

An Analysis and Simulation of Defending Tracking Control Neural Network for Known Affine Discrete-Time Nonlinear Systems Against Adversarial Attacks

Ablasse Kingcaid-Ouedraogo

Last Updated: Feb 3, 2026

1 Abstract

The simulation of a modified tracking control neural network for known affine discrete-time nonlinear systems_[1] will be ported to C++ and optimized. A previous simulation_[1] of this system was designed in matlab. C++ gives lower level control to the programmer, allowing for more granular and programmatic optimizations to be performed. The modification to the tracking control network will to simulate adversarial attacks. A modified system equation that seeks to minimize the impact of this style of a dversarial attack will be designed and simulated.

2 Introduction

System Model

Definition 1. *Affine-In-The-Inputs Discrete-Time Nonlinear System_[1]*

$$\underbrace{x(k+1)}_{\substack{\text{next} \\ \text{system-state}}} = \underbrace{x(k)}_{\substack{\text{current} \\ \text{system-state}}} + \underbrace{\tau}_{\substack{\text{discrete-time} \\ \text{step}}} \underbrace{\left[\underbrace{f(x(k))}_{\substack{\text{state} \\ \text{dynamics}}} + \underbrace{g(x(k))}_{\substack{\text{coupling} \\ \text{dynamics}}} \underbrace{u(k)}_{\substack{\text{control} \\ \text{input}}} \right]}_{\substack{\text{Rate of Change}}} \quad (1)$$

where:

Affine-In-The-Inputs: *The system dynamics are described by the sum of the linear function of the input and the nonlinear function of the state. This property simplifies the analysis and control design.*

Discrete-Time: *The system state is updated at specific, uniform intervals of time.*

x : System state.

f : Nonlinear function that defines the system's internal state dynamics.

g : Nonlinear function that defines the effect of the control input on the state dynamics.

u : The control input applied to the system.

τ : Discrete-time step.

Control Function

Definition 2. For the purpose of finding a solution to the tracking control problem, the control function $u(k)$ has been designed [1]:

$$u(k) = \begin{cases} g(x(k))^{-1}(-f(x(k)) + \Delta r(k) - \beta \text{sgn}(x(k) - r(k))) & \text{if } g(x(k)) \neq 0 \\ 0 & \text{if } g(x(k)) = 0 \end{cases} \quad (2)$$

such that, given a reference trajectory $r(k)$, the system (1) seeks to minimize the closed-loop error dynamics:

Definition 3.

$$e(k+1) = e(k) - \beta \text{sgn}(e(k)) \quad (3)$$

where:

$$\text{sgn}(e(k)) = \begin{cases} 1 & \text{if } e(k) > 0 \\ 0 & \text{if } e(k) = 0 \\ -1 & \text{if } e(k) < 0 \end{cases} \quad (4)$$

and

$$e(k) = x(k) - r(k) \quad (5)$$

Closed-Loop Tracking Control

Definition 4. Substituting $u(k)$ from (2) into the system model (1) gives the closed-loop state equation:

$$x(k+1) = x(k) + \tau \beta \text{sgn}(x(k) - r(k)) + \Delta r(k) \quad (6)$$

where:

$$\Delta r(k) = r(k+1) - r(k) \quad (7)$$

3 Adversarial Attacks

White Noise Attacks

Perturbed Tracking Equation

The mathematical representation of the adversarially perturbed system can be expressed, using equation (1) as follows:

$$x(k+1) = x(k) + \tau [f(x(k)) + g(x(k))u(k)] + I(k) \quad (8)$$

where $I(k)$ represents the adversarial *intrusion* at time step k .

4 Theorems

Existence and Uniqueness of Steady States

Stability and Convergence of Steady States in the Presence of Adversarial Attacks

5 Figures

A functional block-diagram of the controlled nonlinear system described by 1 and 2 is presented in Fig. 1. The diagram includes the blocks of discrete-time summation $+$, sign activation function S , amplification β , nonlinear functions $f(x)$ and $g(x)$, NN solver of system of linear algebraic equations LAESS, reference r , state variable $x(k)$ with initial value $x(1)$, and control input $u(k)$. As one can see in Fig. 1, the NN consists of $3n$ adders, n controlled switches, n amplifiers, n digital differentiators, n blocks of nonlinear functions $f(x)$, $n \times m$ blocks of nonlinear functions $g(x)$, and n NN solvers of a systems of linear algebraic equations.[1]

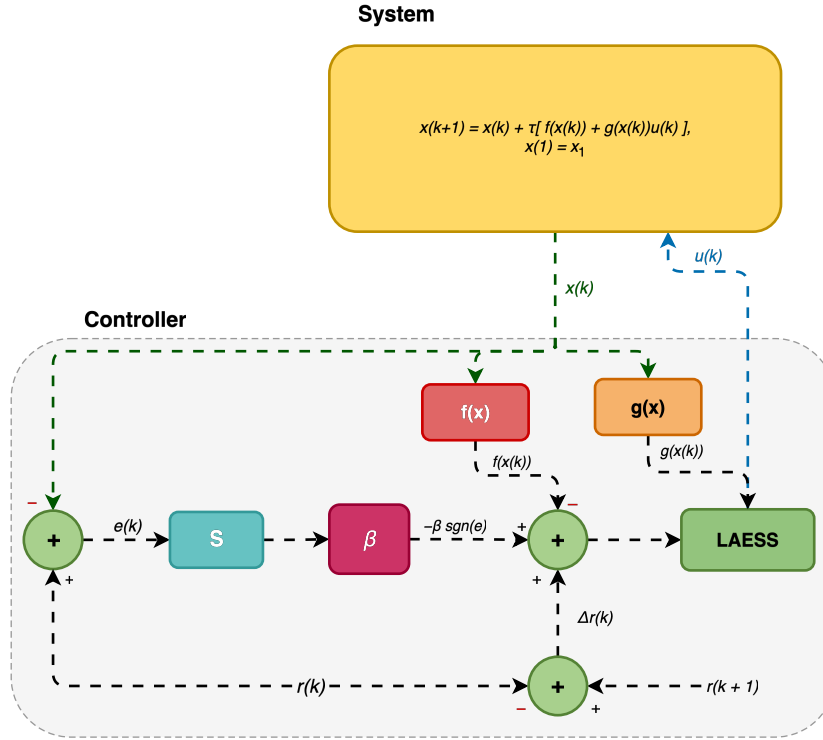


Figure 1: Functional block-diagram of the controlled nonlinear system.

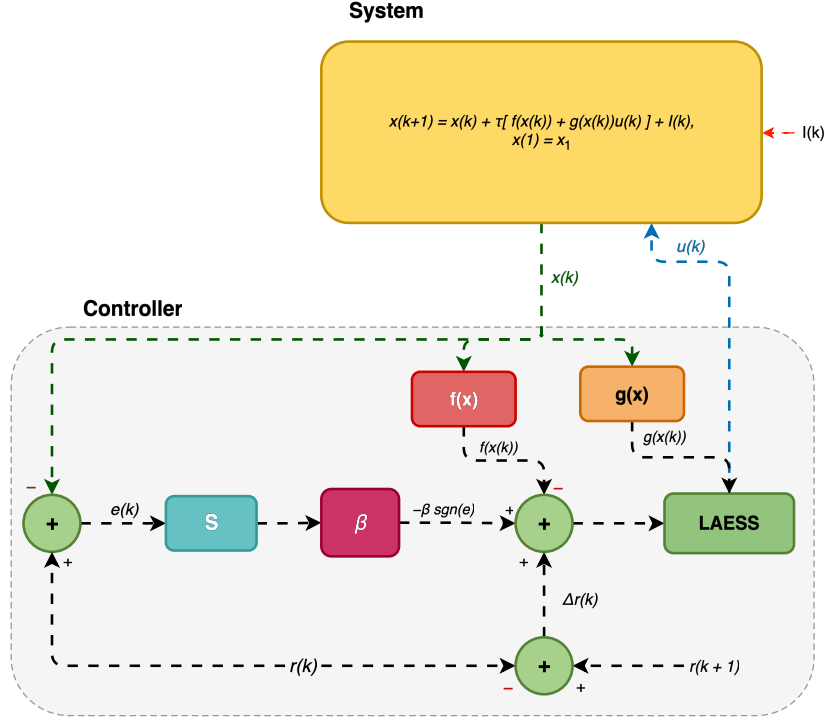


Figure 2: Functional block-diagram of the controlled nonlinear system under adversarial attack.

Figure 3: Functional block-diagram of the controlled nonlinear system with defensive measures.

References

- [1] P. Tymoshchuk, “A tracking control neural network for the known affine in the inputs discrete-time nonlinear systems,” tech. rep., University of North Texas, 2024.
- [2] A. Kingcaid-Ouedraogo, “Analysis of methods of simulation in C++: Migration strategy for tracking control neural network,” 2026. Internal research document, Texas A&M University–Victoria.

A Code Migration: MATLAB to C++

A.1 Motivation

The original tracking control simulation was implemented as a collection of MATLAB scripts (Fig2a.m–Fig2c.m, Fig5.m–Fig5b.m), following the reference implementation in [1]. While MATLAB provides rapid prototyping through built-in matrix operations and plotting, several factors motivated a migration to C++:

1. **Execution speed.** The simulation loop iterates 45,000 time steps with per-step nonlinear dynamics evaluation. C++ compiled code with direct memory access eliminates the interpreter overhead inherent to MATLAB’s execution model.
2. **Memory control.** C++ provides explicit control over stack versus heap allocation, cache-aligned data layouts, and zero-copy data access through pointers—critical when storing $2 \times 45,000$ state matrices.

A.2 Architecture Decisions

The MATLAB implementation uses a procedural style: separate 1-D arrays for each state variable (x , y , $r1$, $r2$, $e1$, $e2$, u) with a single `for` loop driving the simulation. The C++ port uses 2d arrays for the state variables with a single `for` loop as well. It adopts a struct-based architecture that groups related data and separates concerns across compilation units.

A.2.1 Data Layout

Table 1: Data structure mapping between MATLAB and C++ implementations.

MATLAB Variable	C++ Equivalent	Type
$x(1:n)$, $y(1:n)$	<code>stateMatrix</code>	<code>Matrix<double, 2, Dynamic></code>
$r1(1:n)$, $r2(1:n)$	<code>referenceMatrix</code>	<code>Matrix<double, 2, Dynamic></code>
$e1(1:n)$, $e2(1:n)$	<code>velocityErrorMatrix</code> , <code>distanceErrorMatrix</code>	<code>Matrix<double, 2, Dynamic></code>
$r2(i+1)-r2(i)$	<code>deltaReference</code>	<code>Matrix<double, 2, Dynamic></code>
α , β , $\dots$	<code>InternalConditions</code>	<code>struct</code>
n , $t_{\max}$, dt	<code>SimSettings</code>	<code>struct</code>

The key design choice is consolidating per-variable 1-D arrays into $2 \times N$ Eigen matrices. Row 0 stores the velocity (conversion) state x_1 , and row 1 stores the distance (temperature) state x_2 . This layout preserves temporal locality: accessing both states at time step k requires reading two adjacent memory locations in the same column.

A.2.2 Build System

The C++ project uses CMake 3.15+ with `FetchContent` to manage two external dependencies:

Table 2: C++ build dependencies fetched at configure time.

Library	Source	Version	Purpose
Eigen	GitLab	master	Matrix algebra
cxxplot	US Naval Research Lab	v0.4.1	Interactive Qt-based plotting

A.3 Implementation

This section presents side-by-side comparisons of the MATLAB and C++ implementations for each major component of the simulation.

A.3.1 System Dynamics

The CSTR system has two states: conversion x_1 and temperature x_2 . Since the control gain $g_1 = 0$, the conversion state evolves under open-loop natural dynamics. The temperature state uses feedback-linearizing sliding-mode control.

Velocity (conversion) state — open loop. The natural dynamics follow the discretized CSTR model:

$$x_1(k+1) = x_1(k) + \tau \left[-\alpha x_1(k) + D_a(1 - x_1(k)) \exp\left(\frac{x_2(k)}{1 + x_2(k)/\gamma}\right) \right] \quad (9)$$

MATLAB (Fig2b.m, line 29):

```
1 f1(i) = x(i) + dt*(-alpha*x(i) + Da*(1-x(i))*exp(y(i)/(1+
   y(i)/gamma)));
```

C++ (state_space.cpp, lines 24–29):

```
1 stateConditions.stateMatrix(0, i) =
2   prevVelocity +
3   settings.tau *
4   (-internalConditions.alpha * prevVelocity +
5    internalConditions.D * (1 - prevVelocity) *
6    exp(prevDistance / (1 + prevDistance /
      internalConditions.gamma)));
```

The C++ version accesses parameters through the `InternalConditions` struct rather than workspace variables, and uses cached local variables (`prevVelocity`, `prevDistance`) to avoid repeated matrix indexing within the inner loop.

Distance (temperature) state — closed loop. The feedback-linearizing control law cancels the nonlinear dynamics $f_2(x)$, yielding the simplified closed-loop update:

$$x_2(k+1) = x_2(k) + \tau[-\beta \operatorname{sgn}(e_2(k)) + \Delta r_2(k)] \quad (10)$$

where $e_2(k) = x_2(k) - r_2(k)$ is the tracking error and $\Delta r_2(k) = r_2(k+1) - r_2(k)$ is the reference change.

MATLAB (Fig2a.m, line 62):

```
1 y(i+1) = y(i) + dt*(-betta*S(i) + r2(i+1) - r2(i));
```

C++ (state_space.cpp, lines 34–38):

```
1 double deltaR2 = stateConditions.deltaReference(1, i - 1)
  ;
2 stateConditions.stateMatrix(1, i) =
3     prevDistance +
4     settings.tau *
5     (-internalConditions.beta * sgn(e2) + deltaR2);
```

A notable difference is that the C++ implementation pre-computes the Δr_2 matrix in a single Eigen operation before entering the simulation loop, whereas MATLAB computes $r_2(i+1)-r_2(i)$ inline at each step.

A.3.2 Reference Trajectory

The piecewise-constant reference switches between two operating points at $t = 15$ s and $t = 30$ s.

MATLAB uses time-based conditionals:

```
1 if 0<=t(i+1) & t(i+1)<=15
2     r1(i+1) = 0.4472;
3 elseif 15<t(i+1) & t(i+1)<30
4     r1(i+1) = 0.7646;
5 elseif 30<=t(i+1) & t(i+1)<tmax
6     r1(i+1) = 0.4472;
7 end
```

C++ uses index-based ternary expressions:

```
1 stateConditions.referenceMatrix(0, i) =
2     i > 15000 && i < 30000 ? 0.7646 : 0.4472;
```

The index-based approach is valid because $t = i \cdot \tau$ and $\tau = 45/45000 = 0.001$ s, so $t = 15$ corresponds to $i = 15,000$. Additionally, the C++ version pre-computes the reference difference matrix ΔR via Eigen column operations:

```
1 stateConditions.deltaReference =
2     stateConditions.referenceMatrix.rightCols(settings.
3         timeSteps - 1) -
```

```

3      stateConditions.referenceMatrix.leftCols(settings.
        timeSteps - 1);

```

A.3.3 Signum Function

The signum function $\text{sgn}(\cdot)$ is central to the sliding-mode controller. **MATLAB** implements it with explicit branching:

```

1  if (y(i)-r2(i)) > 0
2      S(i) = 1;
3  elseif (y(i)-r2(i)) < 0
4      S(i) = -1;
5  else
6      S(i) = 0;
7  end

```

C++ uses a branchless template:

```

1  template <typename T>
2  int sgn(const T &val) {
3      return (T(0) < val) - (val < T(0));
4  }

```

The C++ implementation avoids conditional branching by exploiting boolean arithmetic: the expression evaluates to +1, -1, or 0 without any **if/else** statements, which can improve pipeline utilization on modern processors.

A.3.4 Visualization

MATLAB's native `subplot/plot` functions were replaced with the `cxxplot` library from the US Naval Research Laboratory. The library provides interactive Qt-based plot windows with a named-parameter API:

```

1  auto vel = cxxplot::plot(timeVector, velStateVector,
2      window_title_ = "Velocity",
3      show_legend_ = true,
4      name_ = "State x(k)",
5      xlabel_ = "Time (s)",
6      ylabel_ = "Velocity");
7  vel.add_graph(timeVector, velRefVector, name_ = "
    Reference r(k)");

```

Table 3 maps the MATLAB plotting calls to their C++ equivalents.

A.4 Performance Comparison

The C++ simulation includes instrumentation via `std::chrono::high_resolution_clock`. The MATLAB simulation uses `tic/toc`. Figure 4 compares execution times across implementations.

Table 3: Visualization API mapping.

MATLAB	C++ (cxxplot)
<code>plot(t, x, 'k-')</code>	<code>cxxplot::plot(t, x)</code>
<code>xlabel('Time (s)')</code>	<code>xlabel_ = "Time (s)"</code>
<code>ylabel('x_1, r_1')</code>	<code>ylabel_ = "Velocity"</code>
<code>legend('x_1', 'r_1')</code>	<code>show_legend_ = true, name_ = ...</code>
<code>subplot(2,1,k)</code>	separate <code>cxxplot::plot()</code> call per window

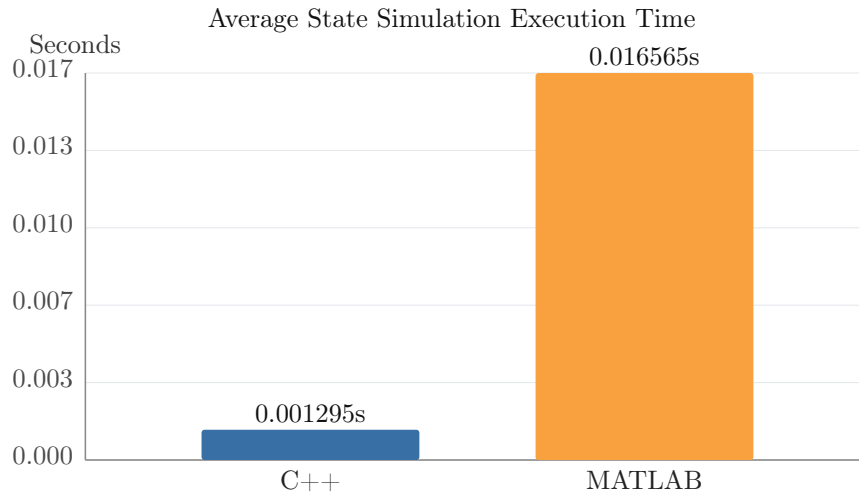


Figure 4: Execution time comparison between MATLAB and C++ implementations of the CSTR tracking control simulation ($n = 45,000$ steps, $\tau = 0.001$ s).

Figure 5 presents side-by-side output from both implementations to verify numerical equivalence.

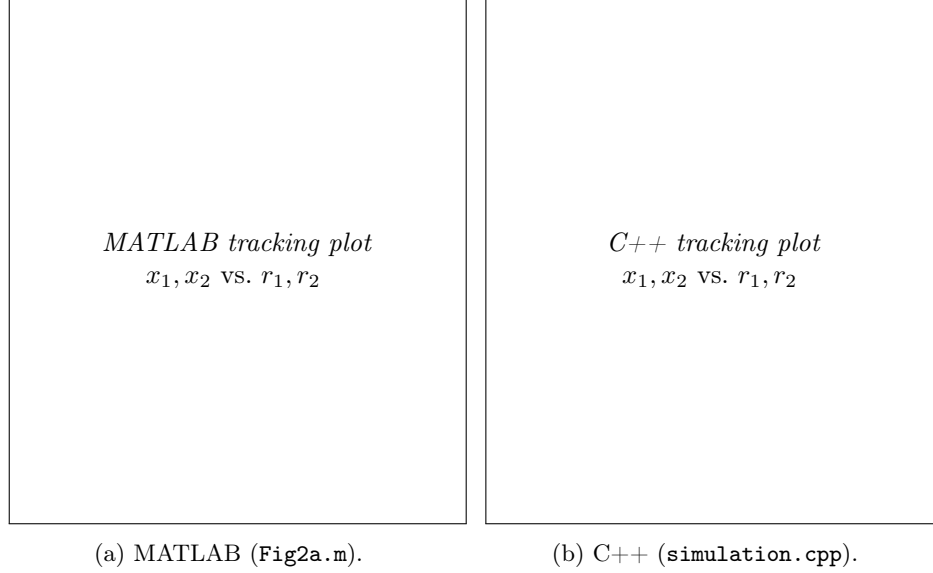


Figure 5: Side-by-side comparison of state trajectories from the MATLAB reference implementation and the C++ port. Both simulations use identical parameters ($\alpha = 1$, $\beta = 100$, $\lambda = 0.3$, $\gamma = 20$, $D_a = 0.072$) and initial conditions $x(0) = \mathbf{0}$.

A.5 Remaining Work

The following components have not yet been migrated from MATLAB to C++:

1. **Fig5 — Mechanical sinusoidal tracking system.** A second-order system with position and velocity states tracking $r(t) = 0.08 \sin(\pi t/2)$, using a cost-based sliding-mode controller with white noise perturbation.
2. **Adversarial attack simulations.** PGD attacks on the temperature measurement channel (`temp_pgd_attack.m`) and neural network approximation-based attacks (`temp_pgd_attack.py` using IBM ART).
3. **Control signal visualization.** MATLAB’s `Fig2b.m` and `Fig2c.m` plot the control input $u(t)$ and tracking errors $e_1(t)$, $e_2(t)$; these are computed in C++ but not yet plotted.
4. **Test infrastructure.** Numerical validation against MATLAB output via CSV export and tolerance-based comparison, as recommended in the migration analysis [2].